Technical Report

Nutrition and Recipe Analytics API

Shuyu Cao
XJCO3011 Web Services and Web Data

April 2026

1 Introduction

This project implements a Nutrition and Recipe Analytics API for the XJCO3011 Web Services and Web Data coursework. The aim was to build a data-driven web service that satisfies the core coursework requirements while also demonstrating design decisions beyond a minimal CRUD application. The final system supports SQL-backed CRUD operations for recipes and ingredients, models many-to-many relationships between recipes and ingredients, and exposes analytical functionality that calculates nutrition summaries and category-level statistics.

The project was intentionally scoped around nutrition and recipe data because this domain supports both transactional operations and analytical queries in a natural way. A single-table application would have been enough to satisfy only the most basic technical requirement, but it would not have demonstrated much database design, API reasoning, or architectural structure. In contrast, the chosen domain makes it possible to discuss relational modelling, validation, authentication, testing strategy, and endpoint design in a technically defensible way during the oral examination.

2 Technology Stack and Rationale

FastAPI was selected as the backend framework because the coursework is explicitly focused on web services rather than server-rendered pages. FastAPI provides strong request and response modelling, automatic validation, and built-in OpenAPI documentation through Swagger UI. This reduced the risk of inconsistencies between the implementation and the documented API, while also making the project easier to demonstrate.

SQLite was chosen as the database because it satisfies the SQL requirement while keeping local execution simple and reliable. For a coursework submission that must be demonstrated live, reduced operational complexity is a practical advantage. SQLAlchemy was used to define database models and support relational queries, Pydantic was used for schema validation and response serialization, and pytest was adopted as the testing framework because it integrates cleanly with FastAPI's test client and supports repeatable endpoint verification.

The main trade-off is that SQLite and a hardcoded API key are not production-grade choices for

a larger deployed service. However, these decisions are appropriate for the coursework context because they prioritize correctness, clarity, and ease of demonstration over production-scale deployment concerns.

3 System Architecture and Data Model

The application follows a layered structure with separate modules for routers, schemas, models, services, and database configuration. This separation improves readability and gives each layer a clear responsibility. Routers define HTTP behaviour, models define persistent entities, schemas define validated input and output structures, and services contain derived logic such as nutrition aggregation. This modular structure was chosen because it supports maintainability and makes the architecture easier to explain during the presentation and question-and-answer session.

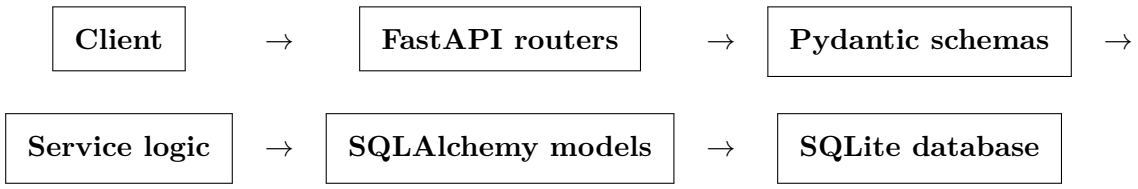

Figure 1: Layered request flow used by the Nutrition and Recipe Analytics API.

The database design uses three entities: `recipes`, `ingredients`, and `recipe_ingredients`. The `recipes` table stores general recipe information such as name, category, difficulty, servings, and instructions. The `ingredients` table stores nutrition information per 100 grams, including calories, protein, fat, carbohydrates, and allergen metadata. The `recipe_ingredients` table models the many-to-many relationship between recipes and ingredients and records the quantity of each ingredient used in a particular recipe.

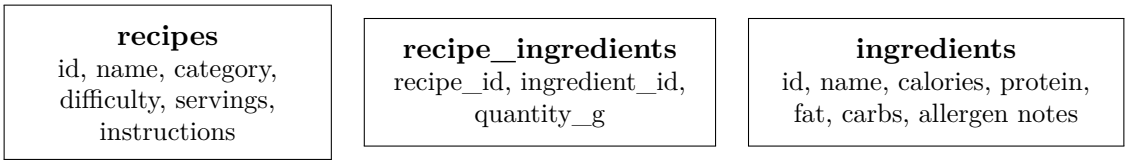

Figure 2: Relational model showing why recipe nutrition can be derived from ingredient quantities.

This design is important because recipe nutrition should not be stored as hardcoded totals. Instead, totals are calculated from ingredient values and quantities. That decision improves consistency and makes the analytical endpoints meaningful. It also demonstrates a more realistic use of relational modelling than a flat CRUD structure would.

4 API Design and Functionality

The API exposes CRUD operations for recipes and ingredients, relationship management endpoints for linking and unlinking ingredients to recipes, and analytics endpoints for nutrition summaries and category counts. Public read endpoints are provided for browsing data, while

write endpoints are protected using an API key passed through the `X-API-Key` header. This authentication model is intentionally lightweight. It introduces an access-control concept without expanding the project into full user registration and session management, which would have added complexity without improving alignment with the coursework goals.

The recipe endpoints support listing, search, retrieval by identifier, creation, update, and deletion. The ingredient endpoints provide the same CRUD structure. In addition, the API supports adding an ingredient to a recipe with a specified quantity and removing an existing recipe-ingredient link. These relationship endpoints are necessary because the nutrition calculations depend on stored composition data rather than directly entered totals.

Two analytical features were added to move the project beyond basic CRUD. The first is a nutrition summary endpoint that calculates total and per-serving calories, protein, fat, and carbohydrates for a given recipe. The second is a category analytics endpoint that counts recipes by category. A search endpoint was also included to support filtered retrieval by category, difficulty, and serving count. Together, these features demonstrate that the API is both data-driven and analytically useful.

5 Testing and Verification

The implementation was developed incrementally with test-first behaviour for each major feature. pytest and FastAPI's `TestClient` were used to validate endpoint responses, status codes, authentication behaviour, and analytics calculations. A dedicated SQLite test database was configured so that tests could run in isolation from local development data. This was important because API tests that share state with manual usage are difficult to trust and difficult to repeat consistently.

The final test suite covers health checks, missing-resource handling, API key enforcement, recipe CRUD, ingredient CRUD, nutrition aggregation, recipe search, category analytics, and removal of ingredients from recipes. At the point of verification, the command `python -m pytest app/tests -v` produced fifteen passing tests. This is relevant to the coursework because the marking criteria explicitly reward evidence of testing and effective handling of errors, not just the presence of runnable endpoints.

6 Challenges and Lessons Learned

One concrete implementation issue arose around route matching. Initially, the static recipe search endpoint conflicted with the dynamic recipe identifier route, which caused the framework to interpret the string `search` as though it were a recipe identifier. The resulting validation failure showed that even small routing decisions can affect API correctness. Reordering the static route before the dynamic route resolved the issue and reinforced the importance of route specificity when designing REST APIs.

Another useful lesson was the value of dependency-based database access. By keeping database sessions behind a shared dependency function, it became straightforward to override the ses-

sion during testing and point the application at a test-specific SQLite database. This reduced coupling between runtime configuration and test execution and made the test suite much more reliable.

7 Limitations and Future Work

The system has several deliberate limitations. Authentication is implemented with a single API key rather than a full user model, and SQLite is used instead of a more scalable production database such as PostgreSQL. Search functionality currently filters by simple recipe-level fields rather than performing richer aggregate filtering such as maximum calories or allergen exclusion across linked ingredients. A PythonAnywhere deployment target has been configured for remote demonstration, but it still uses a lightweight configuration model and requires final package installation on the hosting side before it can be treated as a fully stable hosted service.

Future improvements would therefore focus on three areas. First, authentication could be replaced with JWT-based user accounts and role-based permissions. Second, the search and analytics functionality could be expanded to support richer filtering and reporting over linked ingredient data. Third, the database layer could be migrated from SQLite to PostgreSQL to support a more production-oriented deployment story. These changes would strengthen the project further, but they were intentionally left outside the current scope in order to keep the implementation coherent and demonstrable within the coursework timeline.

8 Version Control, Documentation, and Submission Readiness

The repository has been developed with visible commit history so that the progression of the implementation can be inspected during marking. This is important because the coursework brief explicitly states that examiners will examine repository contents and commit history. The README provides setup instructions, run instructions, testing commands, authentication details, and seed-data usage. API documentation is available through FastAPI's generated OpenAPI interface and has also been captured in a separate document that can be converted into the required PDF submission format.

The supporting submission structure also includes a generated technical report PDF, a generated API documentation PDF, and a presentation deck with matching speaking notes. The presentation material was strengthened with an entity-relationship diagram, a real browser capture of the FastAPI Swagger UI at `/docs`, and example API responses generated from the implemented endpoints. In addition, a PythonAnywhere deployment target has been prepared so that the project can be exposed through a public URL once the remote runtime dependencies are installed.

Key submission links are listed below for direct access:

- GitHub repository: https://github.com/MideasternM/web-services-cw1
- PythonAnywhere deployment target: https://MideasternM.pythonanywhere.com

- Swagger UI target: https://MideasternM.pythonanywhere.com/docs
- API documentation PDF on GitHub: https://github.com/MideasternM/web-services-cw1/blob/feature/nutrition-api/docs/api-documentation.pdf
- Technical report PDF on GitHub: https://github.com/MideasternM/web-services-cw1/blob/feature/nutrition-api/docs/technical-report.pdf
- Presentation deck on GitHub: https://github.com/MideasternM/web-services-cw1/blob/feature/nutrition-api/slides/nutrition-api-deck-updated.pptx
- Supporting slide-source and visual-material directory: https://github.com/MideasternM/web-services-cw1/tree/feature/nutrition-api/submission-src/slides

This means the repository now contains the main components needed for final submission: runnable source code, automated tests, visible version control, API documentation, report content, deployment configuration, and presentation material.

9 Generative AI Declaration

Generative AI was used as an active development aid across planning, architecture exploration, implementation sequencing, debugging support, and documentation drafting. It was used to compare backend stack options, refine the project scope around a domain that could support both CRUD and analytics, structure the implementation into manageable milestones, and improve the written presentation of the design. Final judgement over scope, trade-offs, and acceptance of generated outputs remained with the student. Appendix A provides representative translated excerpts from the development conversation in order to document how GenAI was used during the coursework process.

Appendix A: Representative GenAI Interaction Log

The original development conversation was conducted mainly in Chinese. The excerpts below are concise English translations selected to show requirement checking, technology choice, revision requests, and reflective use of AI rather than passive acceptance of generated output.

1. **Requirement framing and grade target.** The student first asked for a plan aimed at a 70+ outcome and then questioned whether the backend choice needed to follow any specific expectation from teaching, noting that Django had been discussed in class. This interaction showed an attempt to align the implementation with both the marking goal and the module context before committing to a stack.
2. **Technology selection after comparison.** After considering options, the student explicitly chose `FastAPI + SQLite`. This was not a random acceptance of the first suggestion. It reflected a deliberate preference for an API-first framework that would be easier to demonstrate live while still satisfying the SQL-backed coursework requirement.
3. **Submission packaging decisions.** The student then asked for the API documentation and report to be written in LaTeX and also requested a detailed prompt for generating presentation slides. This showed an understanding that a strong submission depended not only on working code but also on presentation quality, documentation quality, and clean packaging of deliverables.
4. **Presentation revision based on the brief.** When the first presentation draft still looked too simple, the student asked for another pass after checking the coursework requirements, specifically noting that the slides should contain stronger visuals and more carefully written text. This prompted a second design cycle focused on assessment fit rather than cosmetic changes alone.
5. **Demand for stronger evidence.** The student asked what the difference was between a real Swagger UI screenshot and the current OpenAPI-style visual, and also questioned whether an ER diagram and API response screenshots were needed. This directly led to replacing schematic visuals with stronger runtime evidence, which improved the academic credibility of the final slides.
6. **Reflective AI declaration request.** In the final stage, the student asked whether the AI record should be placed on the last page of the report and requested wording that would show genuine thinking and learning rather than repetitive agreement. This shaped the appendix into a curated record of decision-making rather than a raw, low-signal transcript.

These excerpts indicate that GenAI was used as a guided assistant rather than as an autonomous author. The student repeatedly checked requirements, questioned implementation choices, requested stronger evidence, and asked for revisions that better matched the coursework criteria. The final scope, architecture, acceptance of generated outputs, and submission decisions remained under student control.